

LLM-Driven Intent Inference and Hierarchical Goal Modeling in Software Engineering

Large Language Models (LLMs) are increasingly being used to understand **what developers intend to achieve** and to assist with multi-step software development tasks. Recent research (2023–2025) has moved beyond simple intent classification in chat sessions, towards representing developer goals as evolving **intention structures** (e.g. trees of sub-goals) and linking those intents to actual code changes. This involves: (1) inferring latent intent from natural-language instructions plus code context, (2) recognizing and planning hierarchical goals (like an *intention tree* of tasks over a coding session), and (3) mining the “why” behind code changes from activity traces (diffs, commits, tests) to build an auditable rationale layer. Below we survey the latest approaches at this intersection, spanning AI pair-programming assistants and version control workflows, with a focus on LLM-centric methods from 2023–2026.

Intent Detection in AI Pair Programming Tools

Modern AI coding assistants (e.g. GitHub Copilot, Replit Ghostwriter, etc.) use LLMs to infer the developer’s intent from natural-language prompts and the current code context. A key challenge is **misalignment** – the code an LLM generates may not match the user’s true intent, leading to back-and-forth clarifications. Studies in 2024 found that users often struggle to articulate nonlinear goals in a linear prompt, and likewise have trouble understanding the LLM’s code output, causing “bidirectional ambiguity” ¹ ². To tackle this, researchers have begun **externalizing the LLM’s internal plan** for the code. For example, *NeuroSync* (UIST 2025) introduces a paradigm of *direct intent-task matching* ³. Instead of the LLM immediately producing code, *NeuroSync* first has the LLM infer a set of coding tasks and their relationships (essentially the LLM’s **understanding of the user’s intent**, structured as a graph of sub-tasks). This inferred task graph is displayed to the user alongside the user’s own stated intents (as a tree) for comparison ³ ². The user can then **edit or re-align the tasks** via a visual interface before any code is generated ⁴ ². By letting developers correct the model’s misunderstanding early, *NeuroSync* significantly improves alignment between generated code and the true intent, reducing frustration and iteration ⁵ ². In essence, it gives the LLM a way to *show its plan* (the “intention tree” of the task) and receive feedback on it prior to coding.

Another line of work focuses on **hierarchical task decomposition** driven by either the user or the LLM. *CoLadder* (UIST 2024) is a system where the *developer* explicitly breaks down a programming task into a hierarchy of sub-tasks using a tree-structured prompt editor ⁶. Each node in the tree is a prompt “block,” and the LLM generates the corresponding code segment for that block ⁶. Developers can manipulate the structure (e.g. promote a block to a higher level to refactor code) and regenerate code accordingly ⁷. A user study found that *CoLadder* helped experienced programmers **externalize their problem-solving intentions** and maintain clarity over the evolving code, from high-level goal to final implementation ⁸. This hierarchical prompt approach ensures the LLM’s output for each sub-problem aligns with the programmer’s mental model of the overall goal ⁸.

LLM researchers have also drawn inspiration from classical AI planning and recent “chain-of-thought” techniques to improve code generation on complex tasks. Multi-stage generation strategies have the LLM **plan before coding**. For example, Han *et al.* (2025) propose a three-stage pipeline: **planning** (generate a solution plan from the NL problem description), **design** (outline a code architecture based

on the plan), and **implementation** (fill in code guided by the plan and design) ⁹ ¹⁰. By breaking a complex problem into a plan and a skeleton before writing code, the LLM's accuracy improved substantially (e.g. ~34% higher pass rate than direct GPT-4 coding on a new benchmark) ¹⁰ ¹¹. Other methods leverage tree-structured reasoning (e.g. *Tree-of-Thought* or Graph-of-Thought) to let the model explore different sub-goals in parallel and choose consistent solutions ¹² ¹³. In summary, **hierarchical planning formalisms** are becoming central to LLM-driven development: by representing a programming task as an *intention tree* of sub-intents, systems can better manage long-range goals and avoid treating each user prompt or code edit in isolation.

Goal Recognition and Rationale in Version Control Workflows

Beyond the IDE, LLMs are being used to infer and document developer intent *in the context of version control* – i.e. commits, pull requests (PRs), and code reviews. One practical application is **automatic commit message generation**. Commit messages ideally describe *what* changed and *why*, but developers often write poor or no messages, leaving future maintainers guessing ¹⁴ ¹⁵. Recent work shows LLMs can fill this gap. Wu *et al.* (ICSE 2025) conducted an empirical study using GPT-style models with in-context learning to generate commit messages from code diffs ¹⁶. They found that prompting LLMs with a few examples of diffs and messages enables them to **generalize** well: the LLM-based approach *outperformed* prior neural models on standard benchmarks, especially in human evaluations of message quality ¹⁶. The LLM's broad knowledge (having seen many code changes during pretraining) helps it capture the *intent behind the change* more effectively than rule-based or template methods, which often “fail to deduce the intent behind code changes” ¹⁷. In practice, GitHub has piloted a similar idea with **Copilot for Pull Requests**, which uses GPT-4 to draft PR descriptions by summarizing the code changes. Developers could insert a marker like `copilot:summary` in a PR, and the AI expands it into a paragraph explaining the changes in that PR ¹⁸. This brings an *intent layer* to version control: instead of a bare diff, there's an AI-generated narrative of what the code is doing. Early versions of Copilot for PRs could even list changed components and link to code, or generate a **“why” explanation** in the form of a poem for fun ¹⁹. While the initial preview was sunset in late 2023, it demonstrated the viability of LLMs for generating high-level descriptions of code edits, and such features are being integrated into enterprise tools ²⁰.

Another active area is **mining the rationale behind code changes** – essentially answering “why was this change made?” using LLMs. Mastropaolo *et al.* (ICPC 2025) developed a pipeline to automatically **extract and generate change rationale** from software repositories ²¹ ²². First, they identified commits where developers had written explicit rationale in the message (e.g. “Fixing X because of Y”) ²³ ²⁴. Using a combination of classifiers and a few-shot LLaMA model, they filtered thousands of commits to build a dataset of ~45K commits paired with explicit rationale sentences ²⁵ ²². They then fine-tuned a code-focused LLM (CodeT5+) to **generate a rationale given the code diff** ²⁶. The results were mixed – the model produced a *meaningful rationale* for a minority of test cases ²⁷ – but this is a promising step. It shows that with training, an LLM can attempt to articulate the underlying reason or justification for a code change, essentially building an **auditable “why” layer** on top of a git diff ²⁷. Even if fully reliable automatic rationales are still out of reach, the released dataset of diff→rationale pairs is spurring further research ²⁸ ²⁹ into tools that **document code changes in human terms**.

LLMs have also been used to classify the *intent or purpose* of code changes, which is useful for planning and quality control. For instance, Islam Opu *et al.* (2025) study **tangled commits**, i.e. single commits that *conflate multiple intents* (e.g. a bug fix *and* an unrelated refactoring in one commit) ³⁰ ³¹. Tangled commits violate best practices and make it hard to audit or revert changes. The authors framed tangled-change detection as a binary classification task on the commit's diff and message. They found that prompting state-of-the-art LLMs (like GPT-4/GPT-5) in a zero/few-shot manner was remarkably

effective – combining the code diff *and* commit message context gave an F1 of ~0.88 in identifying tangled commits ³². In fact, a fine-tuned classifier on LLM-derived embeddings achieved $F1 > 0.90$ ³³. This suggests LLMs can grasp subtle intent distinctions: e.g. “this hunk is a bug fix vs. a refactor” even when intertwined. Such LLM-based analysis could help **disentangle work** by flagging mixed-intent commits and prompting developers to split them. It also hints at *drift detection*: if a commit’s content doesn’t match its purported intent (say, a commit labeled “refactor” actually also adds new features), an LLM can spot that discrepancy.

Toward Persistent Intention Memories and Intent-Aware Assistance

A forward-looking goal is to maintain a **persistent, revisable model of developer intentions** that evolves with the codebase. Instead of treating each prompt or commit in isolation, the assistant would track an “intention tree” over long coding sessions or across many commits. Such an *intention memory* could be updated as the developer’s goals change, detect when work drifts off-track, and provide context-aware help. While an explicit solution isn’t fully realized yet, pieces of this vision are emerging:

- **Intention-awareness in code review and refactoring:** AI code review tools are starting to use higher-level context (design documents, issue trackers, etc.) to understand *why* code is written a certain way, so that suggestions don’t violate the original intent. For example, the CodeRabbit AI reviewer ingests issue descriptions and past PR discussions as part of its context, treating them as *expressions of developer intent* ³⁴. This enables it to catch inconsistencies like a seemingly innocuous change that actually breaks an architectural decision or a team coding convention ³⁵ ³⁶. As one report notes, “*even perfect code can violate design intent... A 'simple' refactor might unintentionally break invisible service boundaries or implicit contracts*” ³⁶. Knowing the intended architecture and invariants (the *why* behind current code) is crucial for AI to recommend safe refactorings. Research prototypes are exploring similar ideas: Guo *et al.* (2025) extract the *reviewer’s intention* from code review comments (e.g. “*rename variable X to Y*”) and feed that to an LLM to perform the code change ³⁷. By translating free-form feedback into a structured intent first, their tool achieved much higher accuracy in applying the correct code refinements ³⁸ ³⁹. This underscores that **explicitly modeling intent leads to more precise code transformations** – a principle future refactoring advisors can build on, so that suggested improvements align with the developer’s goals rather than just generic “code smells.”
- **Drift detection and task management:** With a persistent intention model, an assistant could notice *intent drift* – e.g. if the developer set out to implement feature A, but the code changes are now mostly about fixing an unrelated bug B, the system might ask if the goals have changed or if work should be split. Early clues of this appear in proactive AI assistants. For instance, one field study (JetBrains, 2026) found developers prefer AI interventions at natural breakpoints (like after finishing a task/commit) rather than mid-task interruptions ⁴⁰ ⁴¹. This implies an assistant needs to **recognize task boundaries and switches**. While that study focused on timing suggestions, a logical extension is detecting when the *current task has diverged* from the initial intent. Similarly, LLM-based commit analysis can be extended to multi-commit sequences: by summarizing each commit’s intent, an AI could detect if a series of commits is coherently addressing a stated goal or if extra, unrelated intents crept in. Research on *work item linking* is relevant here – e.g., linking commits to the requirement or issue they resolve. LLMs can assist by reading an issue description (the planned intent) and a set of commit diffs, and determining which changes satisfy that intent and which do not. Although robust “intention drift detectors” are not yet standard, these building blocks suggest it’s feasible.

- **Evolving intention trees:** Maintaining a *long-lived intention tree* means the assistant continuously updates a hierarchy of what the developer is working on (and why) as the code evolves. For example, at the start of a project, the root might be “Implement Feature X,” which breaks down into sub-tasks A, B, C. If mid-way the developer decides to refactor a module (not originally planned), the assistant can incorporate that as a new branch in the intention tree (or flag it as a tangent). There is early discussion in the community about how an IDE-integrated LLM might serve as a kind of *project historian* – tracking decisions and intents over months. While no system fully realizes this yet, the concept aligns with **HTN (Hierarchical Task Network)** planning in classical AI, now applied to interactive coding. Some experimental tools (e.g. Copilot’s prototypes like “Copilot Workspace” mentioned by GitHub ⁴² ⁴³) hint at this direction: they attempted to generate a plan for large-scale codebase changes (like “replace library X with Y” – a goal that spans many files) and even automatically create the PR with all the required edits. Those systems essentially **decompose a high-level intent into actionable edits** across a codebase ⁴² ⁴³ . As these features mature, we can expect the AI to maintain an internal representation of such goals and monitor progress (e.g. know which sub-tasks are done, which are pending or blocked).

In summary, the latest research (2023–2026) demonstrates a clear trend: **LLMs are not just coding from prompts, but increasingly reasoning about developer intentions** – structuring them into plans, aligning code to those plans, and explaining the rationale behind changes. Academic prototypes like CoLadder and NeuroSync show the power of making the *intention graph* explicit and editable ⁴ ⁶ . Meanwhile, tools in the version control domain use LLMs to connect code edits with natural-language descriptions and reasons, from commit summaries ¹⁶ to commit rationales ²⁶ . Although a fully persistent “intention memory” that spans an entire project is still an aspirational goal, these advances lay the groundwork. By attributing intents to concrete code edits and maintaining an evolving intention tree, future development environments could provide intelligent support that not only writes code, but also *understands why* the code is being written – catching misalignment early, untangling mixed objectives, and proactively guiding refactoring and design decisions in an **intent-aware** manner. This is a young but rapidly evolving area, and we can expect both research and industry to continue bridging the gap between a developer’s mind and the code they produce, with LLMs as the key enabler.

Sources:

- W. Zhang *et al.* (2025). “NeuroSync: Intent-Aware Code-Based Problem Solving via Direct LLM Understanding Modification.” UIST 2025 ¹ ²
- R. Yen *et al.* (2024). “CoLadder: Manipulating Code Generation via Multi-Level Blocks.” UIST 2024 ⁶ ⁸
- Y. Han *et al.* (2025). “Multi-stage guided code generation for Large Language Models.” Eng. Appl. of AI, 139(2025) ⁹ ¹⁰
- Y. Wu *et al.* (2025). “An Empirical Study on Commit Message Generation using LLMs via In-Context Learning.” ICSE 2025 (to appear) ¹⁶ ¹⁷
- GitHub Next (2023). “Copilot for Pull Requests” – technical preview overview ¹⁸
- A. Mastropaolo *et al.* (2025). “Towards Generating the Rationale for Code Changes.” ICPC 2025 ²⁶ ²⁷

- M.N.I. Opu *et al.* (2025). *"LLM-Based Detection of Tangled Code Changes."* arXiv preprint 2505.08263
44 33
- CodeRabbit Engineering Blog (2023). *"AI-Powered Code Reviews with LanceDB (Case Study)"* 35 36
- Q. Guo *et al.* (2025). *"Intention is All You Need: Refining Your Code from Your Intention."* arXiv 2502.08172 37 45
- N. Kuo *et al.* (2026). *"Developer Interaction Patterns with Proactive AI: A Five-Day Field Study."* IUI 2026 40 41

1 2 3 4 5 12 13 [2508.02823] NeuroSync: Intent-Aware Code-Based Problem Solving via Direct LLM Understanding Modification
<https://arxiv.labs.arxiv.org/html/2508.02823v1>

6 7 8 CoLadder: Manipulating Code Generation via Multi-Level Blocks
<https://www.jeffjianzhao.com/papers/coladder.pdf>

9 10 11 Multi-stage guided code generation for Large Language Models - ScienceDirect
<https://www.sciencedirect.com/science/article/abs/pii/S095219762401649X>

14 15 16 17 An Empirical Study on Commit Message Generation using LLMs via In-Context Learning
<https://arxiv.org/html/2502.18904v1>

18 19 20 42 43 GitHub Next | Copilot for Pull Requests
<https://githubnext.com/projects/copilot-for-pull-requests>

21 22 23 24 25 26 27 28 29 antoniomastropaolo.com
https://antoniomastropaolo.com/assets/pdf/ICPC_RENE_2025.pdf

30 31 32 33 44 LLM-Based Detection of Tangled Code Changes for Higher-Quality Method-Level Bug Datasets
<https://arxiv.org/html/2505.08263v3>

34 35 36 Case Study: How CodeRabbit Leverages LanceDB for AI-Powered Code Reviews
<https://lancedb.com/blog/case-study-coderabbit/>

37 38 39 45 arxiv.org
<https://arxiv.org/pdf/2502.08172>

40 41 Developer Interaction Patterns with Proactive AI: A Five-Day Field Study
<https://arxiv.org/html/2601.10253v1>